Nome: Bianca Banaletti e Sabrina Figueiredo
Disciplina: Sistemas Operacionais II

Resumo da aula do dia 12/08

1. Condições de corrida,
2. Regiões críticas
3. Exclusão mútua.

Condições de corrida: Quando dois ou mais processos compartilham um mesmo recurso, devem existir mecanismos de controle para evitar esses tipos de problemas, conhecidos como condições de corrida. Uma condição de corrida em sistemas operacionais ocorre quando dois ou mais processos ou threads acessam e modificam um recurso compartilhado ao mesmo tempo.

Regiões Críticas: A região crítica é a parte do código de um processo em que ocorre o acesso a um recurso compartilhado, como um arquivo, uma variável, uma área da memória ou um banco de dados. Como vários processos podem tentar acessar esse recurso ao mesmo tempo, é necessário garantir que apenas um deles execute sua região crítica por vez. Toda vez que um processo deseja executar instruções de sua região, obrigatoriamente deverá executar antes um protocolo de entrada nessa região crítica. Da mesma forma, ao sair da região crítica um protocolo de entrada deverá ser executado. Os protocolos de entrada e saída garantem a exclusão mútua da região crítica de um programa.

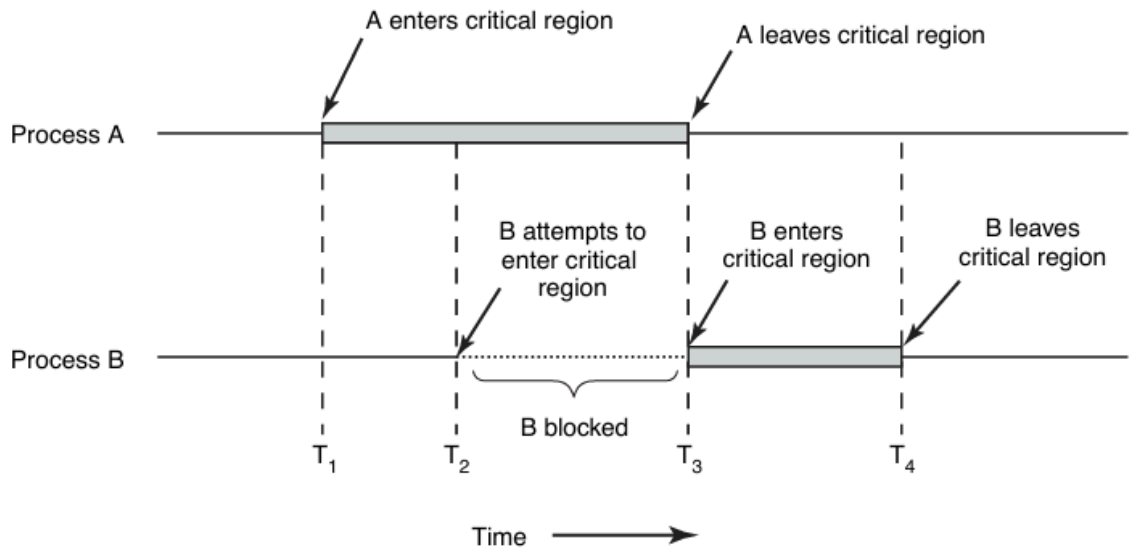


Figure 2-22. Mutual exclusion using critical regions.

Exclusão Mútua: A exclusão mútua é o mecanismo que garante que dois ou mais processos não executem suas regiões críticas simultaneamente quando elas acessam o mesmo recurso compartilhado. O objetivo é evitar problemas causados pelo acesso concorrente. Por exemplo, se dois processos tentarem atualizar simultaneamente o saldo de uma mesma conta, um processo pode sobrescrever a alteração feita pelo outro, causando um resultado incorreto.

Assim, quando um processo está utilizando o recurso, os demais precisam aguardar. Quando o primeiro termina, ele libera o recurso, permitindo que outro processo o utilize. Além de garantir a exclusão mútua, uma solução adequada deve evitar principalmente:

Starvation (espera indefinida): ocorre quando um processo fica esperando indefinidamente para acessar o recurso.

Bloqueio desnecessário: ocorre quando um processo que não está mais utilizando sua região crítica continua impedindo outros processos de utilizarem um recurso que está livre.

Uma solução para reduzir o problema de starvation é utilizar uma fila FIFO, na qual os processos são atendidos na ordem em que solicitaram o recurso.

As soluções de software implementam a exclusão mútua utilizando algoritmos e variáveis compartilhadas, sem depender diretamente de instruções especiais do hardware. Um exemplo clássico é o algoritmo de Peterson, desenvolvido para dois processos. Ele utiliza variáveis para indicar: qual processo deseja entrar na região crítica; qual processo deve ter a preferência quando ambos tentam entrar ao mesmo tempo.

Outras soluções históricas incluem os algoritmos de Dekker e Lamport.Essas soluções são importantes para compreender os conceitos de sincronização, embora os sistemas operacionais modernos normalmente utilizem mecanismos de hardware para implementar primitivas de sincronização de forma mais eficiente.

As soluções de hardware utilizam instruções especiais do processador que permitem realizar determinadas operações de forma atômica, ou seja, sem que outro processo consiga interferir entre as etapas da operação. Entre os mecanismos mais conhecidos estão:

- Test-and-Set: verifica e altera uma variável de bloqueio em uma única operação atômica.
- Compare-and-Swap: compara o valor de uma variável com um valor esperado e, se forem iguais, realiza uma alteração.
- Swap: troca, de maneira atômica, os valores de duas variáveis.

Essas instruções podem ser utilizadas para implementar mecanismos de sincronização, como mutexes, semáforos e spinlocks. Por exemplo, um processo pode tentar adquirir um bloqueio antes de entrar na região crítica. Se o bloqueio estiver ocupado, ele espera. Quando o processo que possui o bloqueio termina, ele o libera.